This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3665382

Multimodal Anomaly Detection for Microservice Systems via Grassmann Manifolds-based Graph Fusion

Shiming He, Keyao Feng, Kaixuan Meng, Kun Xie *Member, IEEE*, and Xibin Zhao *Senior Member, IEEE*

***Abstract*—Microservice architecture decomposes complex applications into multiple small, independent services, enabling independent deployment and scaling. However, microservice systems introduce complex and dynamic interactions between service instances. When a service instance fails, it can significantly degrade the performance of the entire system, harm the user experience, and cause substantial economic losses. Therefore, effective anomaly detection for service instances is crucial to ensure system reliability. Metrics, logs, and traces provide complementary insights into microservice operations, and recent approaches have attempted to jointly exploit these modalities for anomaly detection. However, the varying data volumes per interval and heterogeneous data types across modalities, combined with the complex and elusive relationships between them, make effective representation and learning challenging. To address these limitations, we propose MGFusion, an unsupervised, end-to-end multimodal anomaly detection method for microservice systems using Grassmann manifolds-based graph fusion. MGFusion employs a robust method to unify metrics, logs, and traces into time series, enabling consistent processing across modalities. It leverages multiple graph structure learning (GSL) techniques to explore multi-view relationships between the modalities. To mitigate the impact of data noise, we further introduce prior knowledge based on the Adamic-Adar Index (AAI) and employ a Grassmann manifolds-based graph fusion method to combine multiple basic graph structures. Finally, anomaly detection is achieved using Diffusion Convolutional Recurrent Neural Network (DCRNN) predictors and anomaly score calculation. Extensive experiments on a public dataset demonstrate that MGFusion effectively fuses multimodal data and captures complex relationships, significantly improving the accuracy and efficiency of anomaly detection. Compared to the state-of-the-art unsupervised multimodal anomaly detection method, AnoFusion, MGFusion achieves an improvement in the F1 score ranging from 11.1% to 15.3%.**

***Index Terms*—Anomaly Detection, Multimodal Data, Graph Structure Learning, Graph Fusion, Grassmann Manifolds.**

I. Introduction

MICROSERVICE architecture has emerged as the dominant paradigm in modern software system design due to its modularity, scalability, and ease of maintenance. By decomposing complex applications into small, independently deployable services, the microservice architecture enhances system flexibility and maintainability. Instances (e.g., virtual machines or containers) serve as operational entities of these services.

Microservice systems involve intricate and dynamic interactions among instances, including service invocations and resource competition [1], [2]. Anomalies in instances significantly degrade system performance, adversely impacting user experience, and causing potential economic losses. Instance anomaly detection, which identifies Anomalies or abnormal behaviors of microservice instances by analyzing monitoring data, is essential to ensure service reliability [3].

Microservice systems collect three primary types of status data [4]: metrics, logs, and traces, as illustrated in Fig. 1. Metrics are quantitative measures of system performance, typically presented as time series. Logs are semi-structured data generated by "printf" statements or logging frameworks, recording key system events [5]. Traces capture complete invocation chains between services, with each invocation represented as a "span" [6].

Fig. 1 shows an example of these data types. Metrics include CPU usage rate, memory usage, and throughput, etc. Logs contain timestamped entries with levels (e.g., ERROR) and messages (e.g., "Try to get service's inst, retry for 1 time ..."). Traces include service caller, callee, timestamp, response time (RT), and status codes. For instance, at timestamp 1625104624 (2021-07-01 09:57:04), the RT from service S3 to S4 is 10 seconds, with a status code of 200 (success). At timestamp 1625104625, the RT from S2 to S4 is 15 seconds, but the status code is 500 (server error). These multimodal data offer complementary insights into the system's status, making them invaluable for anomaly detection.

While single-modal anomaly detection methods [7]–[10] have achieved some success, they are inherently limited. Anomalies often manifest differently across modalities, as shown in Table I. For example, CPU contention is detectable via metrics but not logs or traces. Conversely, code logic errors are reflected in logs and traces but not metrics. No single-modal data can cover all anomaly types. Thus, multimodal analysis is essential for comprehensive and accurate anomaly detection.

Data fusion is at the core of multimodal anomaly detection. Based on the fusion stage, existing methods fall into three categories: late fusion, early fusion, and intermediate fusion [11]. Late fusion [12]–[14] combines independent detection results from each modality using voting or weighting. Early fusion [15]–[20] converts multimodal data into a unified

This work is supported in part by the National Natural Science Foundation of China under Grants 62025201, and 62272062, the Science and Technology Innovation Program of Hunan Province under Grant 2023RC3139, and the Natural Science Foundation of Hunan Province 2025JJ50373. (*Corresponding author: Kun Xie*)

Shiming He, Keyao Feng, and Kaixuan Meng are with the School of Computer Science and Technology, Changsha University of Science and Technology, the Hunan Provincial Key Laboratory of Intelligent Processing of Big Data on Transportation, Changsha, 410114, China (e-mail: smhe_cs@csust.edu.cn; kyfeng@stu.csust.edu.cn; kaixuan@stu.csust.edu.cn).

Kun Xie is with the College of Computer Science and Electronics Engineering, Hunan University, the Ministry of Education Key Laboratory of "Fusion Computing of Supercomputing and Artificial Intelligence", Changsha, 410082, China (e-mail: xiekun@hnu.edu.cn).

Xibin Zhao is with the School of Software, Tsinghua University, Beijing, 100084, China (e-mail: zxb@tsinghua.edu.cn).

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:31:17 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3665382

TABLE I: Multimodal data abnormal modes for different anomalies ('—' indicates no detectable abnormality).

Anomaly Type	Metric	Log	Trace
CPU Contention	High CPU usage	—	—
Code Logic Error	—	Error/Warning	Changed calling relationship
Network Interruption	Error rate increases	Error/Warning	Latency increases
Network Jam	Throughput decreases	—	—
Configuration Error	—	Error/Warning	Call interrupt

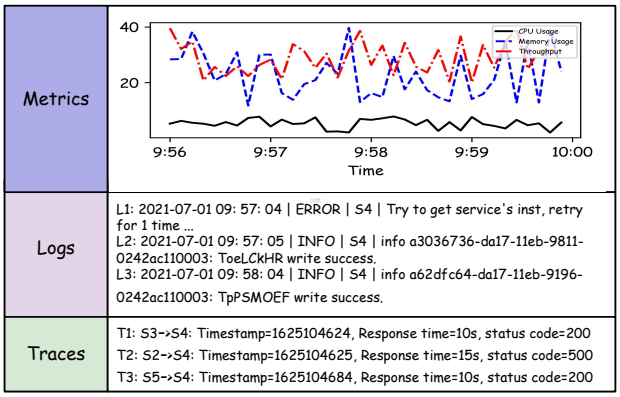


Fig. 1: Multimodal status data. The metrics fail to capture anomalous behaviors in this case because the example represents a configuration error, as shown in Table I. Such anomalies are exclusively reflected in logs and traces, rather than in metrics.

format, while intermediate fusion [21]–[24] extracts features independently for each modality and combines them.

Early fusion has gained prominence due to its ability to capture inter-modal interactions early in the processing pipeline. The core steps in early fusion-based multimodal anomaly detection consist of unified modality conversion, relationship learning, and graph-based anomaly detection. To illustrate this process, we take the conversion of data into a unified time series format as an example, as illustrated in Fig. 2. In this approach, multimodal data are transformed into time series and relationships between time series are modeled for anomaly detection using Graph Neural Networks (GNNs). However, early fusion faces several challenges:

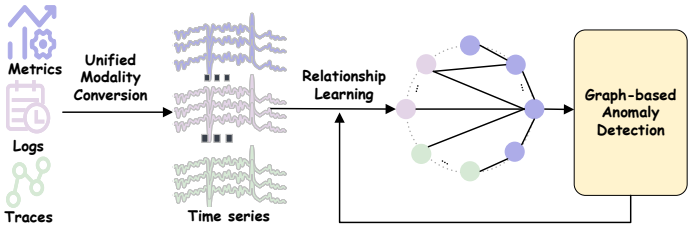


Fig. 2: Key processes of early fusion-based multimodal anomaly detection, with the unified conversion into time series as an example.

- **Inconsistent Data Volume and Type in Time Series Conversion.** Conversion should align multimodal data to a unified time interval. Multiple records may appear in a single interval, and the data volume per interval and data type often varies across modalities. While continuous data can be averaged, event-based data (e.g., logs or traces) require more nuanced handling to preserve semantic integrity. Naive averaging of discrete data, such as status codes, can lead to meaningless results. For example, for spans T1 and T2 in the same interval (such as one minute), a status code average of 350 (redirection) from 200 (success) and 500 (server error) is meaningless. Some works [19], [20] ignore them directly, and some works [18] retain only the value of the first span, discarding critical information (500 server error).
- **Noise in Raw Data Adversely Affects Relationship Learning.** Accurate modeling of inter-time series dependencies forms the foundation of graph-based anomaly detection. Current approaches typically mine these relationships guided by raw data. Noise in raw data, which includes errors, interference, or loss that may occur during the processes of data collection, transmission, and storage, can mislead relationship learning and degrade the performance of graph-based anomaly detection models.
- **Limitations of Existing Relationship Learning Methods.** Current methods often fail to capture complex, multi-view dependencies across modalities. For example, mutual information-based methods [18] can handle linear correlations but struggle with nonlinear or hierarchical dependencies. Graph structure learning (GSL) in single-modal anomaly detection learns relationships among homogeneous nodes (e.g., metrics) but cannot generalize to heterogeneous node types (e.g., cross-modal interactions) directly.

To address these challenges, we propose MGFusion, a multimodal anomaly detection framework for microservice systems based on Grassmann manifolds and graph structure learning. The key innovations of MGFusion include:

- **Robust Time Series Conversion:** MGFusion statistically aggregates continuous data, and employs frequency encoding to convert discrete data while preserving critical information, addressing inconsistencies in data type.
- **Noise-Resilient Relationship Learning:** MGFusion leverages the Adamic-Adar Index similarity metric [25] to guide relationship learning, reducing the impact of noise by emphasizing meaningful node similarities.
- **Grassmann Manifolds-Based Graph Fusion:** MGFusion learns multiple basic graph structures using different graph structure learning methods and combines them via Grassmann manifolds-based fusion to preserve unique structural properties and model complex and multi-view interdependencies.

Experimental results on public datasets demonstrate that MGFusion outperforms nine state-of-the-art single-modal and multimodal anomaly detection methods. For example, MGFusion improves the F1 score by 11.1% to 15.3% compared to AnoFusion.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:31:17 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3665382

The remainder of this paper is organized as follows. Section II reviews related work. Sections III to VII detail the MG-Fusion framework and its components. Section VIII evaluates the proposed method and Section IX concludes the paper.

II. Related Work

In this section, we summarize the related work on single-modal and multimodal anomaly detection, highlighting key methods, and their limitations.

A. Single-Modal Anomaly Detection

Single-modal anomaly detection focuses on analyzing individual data modalities such as metrics, logs, or traces to identify anomalies. Each modality provides unique insight but is inherently limited in its ability to cover all possible anomaly types.

*1) **Metric-Based Anomaly Detection**:* Metrics, typically represented as time series, exhibit both intra-time series (within individual metrics) and inter-time series (between different metrics) relationships. To effectively capture these relationships, graph-based methods [26], [27] have been widely adopted. GSL learns an optimal graph structure jointly with the downstream anomaly detection task. GDN (Graph Deviation Network) [28] is a pioneering GSL-based method that learns graph structures using learnable node embeddings. Subsequent works, such as FuSAGNet [29], GTA [30], and FuGLAD [31], extend this idea by introducing sparse or multiple graph structures for better anomaly detection performance.

*2) **Log-Based Anomaly Detection**:* Log anomaly detection primarily employs sequence-learning models like Long Short-Term Memory (LSTM) networks or Transformers. These models learn normal sequence patterns and predict the next log template in an unsupervised manner. If the actual template is not among the top k predictions, it is flagged as an anomaly. Representative methods include: DeepLog [32], LogAnomaly [33], and Logsy [34]. Supervised or semi-supervised methods, such as LogRobust [35] and NeuralLog [36], learn to distinguish between normal and abnormal sequences. LogBP-LoRA [37] combines pre-trained models with parameter-efficient fine-tuning (LoRA) to enhance detection efficiency.

*3) **Trace-Based Anomaly Detection**:* Trace anomaly detection captures patterns in invocation chains or key performance indicators (KPIs) of system traces. TraceAnomaly [38] represents each trace's response time and invocation path as feature vectors. TraceStream [39] uses tree-based dynamic tracking vectors to extract both structural and temporal features. CRISP [6] constructs key paths in a top-down and bottom-up manner to identify services impacted by end-to-end delays.

While effective in specific cases, single-modal anomaly detection methods are limited by their reliance on one type of data, making them unable to detect all anomaly types.

B. Multimodal Anomaly Detection

Multimodal anomaly detection leverages the complementary strengths of metrics, logs, and traces to improve detection accuracy. Based on the stage of data fusion, multimodal methods are categorized into late fusion, early fusion, and intermediate fusion, as shown in Fig. 3.

*1) **Late Fusion**:* Late fusion was among the earliest approaches in multimodal anomaly detection. In this approach, each modality is processed independently and the results are combined using voting or weighting. PDiagnose [12], MADMM [13], and MicroCBR [14] follow this approach. However, late fusion methods fail to model the interactions and relationships between different modalities, limiting their ability to capture complex cross-modal dependencies.

*2) **Early Fusion**:* Early fusion combines multimodal data at the preprocessing stage by converting them into a unified format, such as events or time series.

- **Events:** DeepTraLog [15] represents traces and logs as invocation chain events. DiagFusion [16] constructs an event dependency graph using embedding and data augmentation for anomaly classification. Nezha [17] builds an event graph to locate anomalies, providing a graphical explanation of root causes.
- **Time Series:** AnoFusion [18] converts multimodal data into multivariate time series and constructs a multimodal graph using mutual information to generate six adjacency matrices for different modalities. MSTGAD [19] builds a microservice graph from traces after converting multimodal data into time series. MRCA [20] uses variational autoencoders (VAE) to learn normal patterns from multimodal data converted into time series.

In summary, events-based approaches are better suited for root cause analysis due to their discrete nature, while time series-based approaches are more effective for anomaly detection as they preserve temporal ordering. This ordering enables the identification of evolving patterns and deviations over time. Therefore, our method employs early fusion, converting multimodal data into time series for anomaly detection.

*3) **Intermediate Fusion**:* Intermediate fusion combines features extracted independently from different modalities. SCWarn [21] uses LSTM to learn the temporal dependencies in multimodal data. UAC-AD [22] combines modality-specific features in a shared space. Hades [40] employs dual attention layers to extract both cross-modal interactions and intra-modal dependencies. Eadro [24] utilizes Hawkes processes and dilated causal convolution (DCC) layers to integrate multimodal features. SIADA [23] fuses multimodal features using context-aware multigraph representation and recurrent encoders.

III. Problem Definition and Solution Overview

This section formally defines the problem of multimodal anomaly detection in microservice instances, provides the solution formulation, and presents an overview of our proposed approach.

A. Problem Definition

The goal of multimodal instance anomaly detection is to identify abnormal behaviors at specific timestamps by analyzing multimodal status data collected from a microservice instance. Unlike methods that focus on anomaly detection across multiple service instances, our approach is specifically tailored to detect anomalies within a single instance at precise moments in time.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:31:17 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3665382

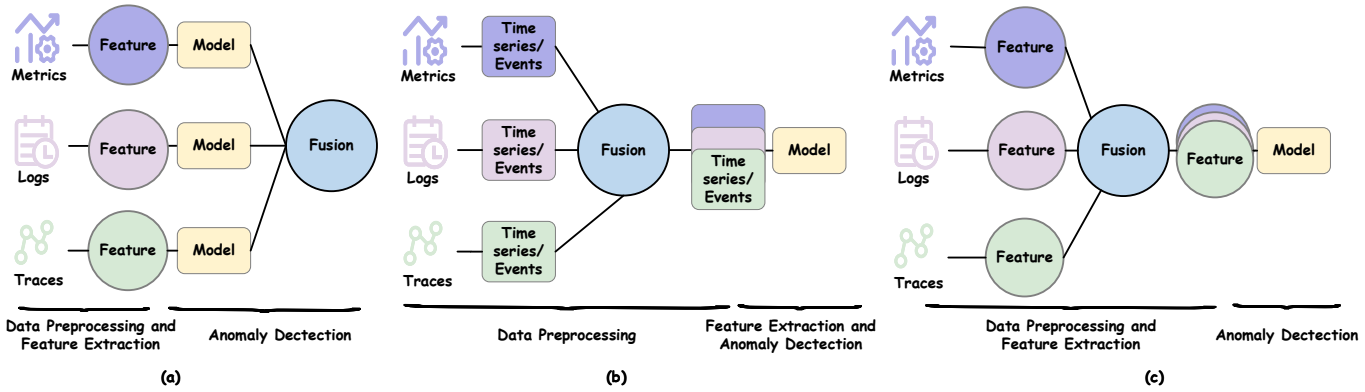


Fig. 3: Multimodal Fusion Strategies: (a) Late Fusion, (b) Early Fusion, (c) Intermediate Fusion.

We assume that all collected status data for a given instance are time-synchronized. Metrics are collected at regular intervals. Logs are generated as event-triggered messages. Traces are recorded when user requests invoke service interactions.

The multimodal data collected over N time intervals is denoted as $Data_M$, $Data_L$, and $Data_T$ for metrics, logs, and traces, respectively. At each timestamp t, the anomaly ground truth is represented as y_t, where $y_t = 1$ indicates an anomaly and $y_t = 0$ indicates normal behavior.

Given historical data within a sliding window w, denoted as $Data_M[t-w,t]$, $Data_L[t-w,t]$, and $Data_T[t-w,t]$, the anomaly detection task identifies whether an anomaly exists at timestamp t, denoted as $\hat{y}_t$. This process is formalized as:

$$\hat{y}_t = F(Data_M[t-w,t], Data_L[t-w,t], Data_T[t-w,t]) \tag{1}$$

where F represents the anomaly detection model.

B. Solution Formulation

To fully exploit the benefits of early fusion, we adopt a multimodal anomaly detection paradigm that combines time series conversion and graph structure learning. Before presenting the solution overview, we formulate the key components of the process.

*1) **Serialization of Multimodal Data**:* As illustrated in Fig. 2, we convert all modalities into time series and combine them into a unified multivariate time series. The conversion process can be formalized as follows:

$$\begin{aligned} X_{\text{Metrics}} &= \text{TSM}(Data_M) \in \mathbb{R}^{K_M \times N}, \\ X_{\text{Logs}} &= \text{TSL}(Data_L) \in \mathbb{R}^{K_L \times N}, \\ X_{\text{Traces}} &= \text{TST}(Data_T) \in \mathbb{R}^{K_T \times N}, \end{aligned} \tag{2}$$

where $\text{TSM}()$, $\text{TSL}()$, and $\text{TST}()$ denote the time series conversion functions for metrics, logs, and traces, respectively. K_M, K_L, and K_T represent the number of converted time series for metrics, logs, and traces, respectively, and N is the total number of time intervals.

The combined multivariate time series is represented as:

$$X = \begin{bmatrix} X_{\text{Metrics}} \\ X_{\text{Logs}} \\ X_{\text{Traces}} \end{bmatrix} \in \mathbb{R}^{K \times N}, \tag{3}$$

where $K = K_M + K_L + K_T$ is the total number of converted time series. At each timestamp t, $x_t \in \mathbb{R}^K$ represents the multivariate data. For a historical window of size w, the sub-time series at timestamp t is represented as:

$$X_t = [x_{t-w}, x_{t-w+1}, \ldots, x_{t-1}] \in \mathbb{R}^{K \times w}.$$

The anomaly detection task converts to determine whether an anomaly occurs at timestamp t, given the historical data X_t.

*2) **Graph Construction**:* GNNs have proven effective for multivariate time series anomaly detection. To leverage GNNs, we construct a graph $G = (V, E)$ for the converted multivariate time series with the following components:

- **Nodes** (V): Each node represents a converted time series, with the total number of nodes given by $|V| = K$.
- **Edges** (E): Edges represent hidden relationships between the converted time series.

Since the relationships between time series are unknown, we employ GSL to dynamically learn an adjacency matrix $A \in \mathbb{R}^{K \times K}$. Each element $A^{(i,j)}$ indicates the presence of a connection between nodes i and j, where $A^{(i,j)} = 1$ denotes an edge, and $A^{(i,j)} = 0$ denotes no edge. The adjacency matrix captures dependency relationships that are critical for accurate anomaly detection.

*3) **Anomaly Detection**:* Using the learned graph structure A, GNNs model inter-time series relationships to facilitate anomaly detection. The detection process involves the following steps: GSL dynamically learns the adjacency matrix A. Given the historical sub-time series X_t and the learned graph structure A, the model predicts the value of x_t at timestamp t, denoted as $\hat{x}_t \in \mathbb{R}^K$. An anomaly score s_t is calculated based on the difference between the predicted value $\hat{x}_t$ and the actual value x_t. An anomaly is detected at timestamp t if the anomaly score s_t exceeds a predefined threshold.

The anomaly detection process can be formally defined as:

$$\begin{aligned} A &= \text{GSL}(X), \\ \hat{x}_t &= \text{f}(X_t, A), \\ s_t &= \varphi(x_t, \hat{x}_t), \\ \hat{y}_t &= \begin{cases} 1, & \text{if } s_t > T, \\ 0, & \text{if } s_t \leq T, \end{cases} \end{aligned} \tag{4}$$

where A is the learnable adjacency matrix, $\text{GSL}()$ represents the graph learning function, $\text{f}()$ is the prediction function, $\varphi()$

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:31:17 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3665382

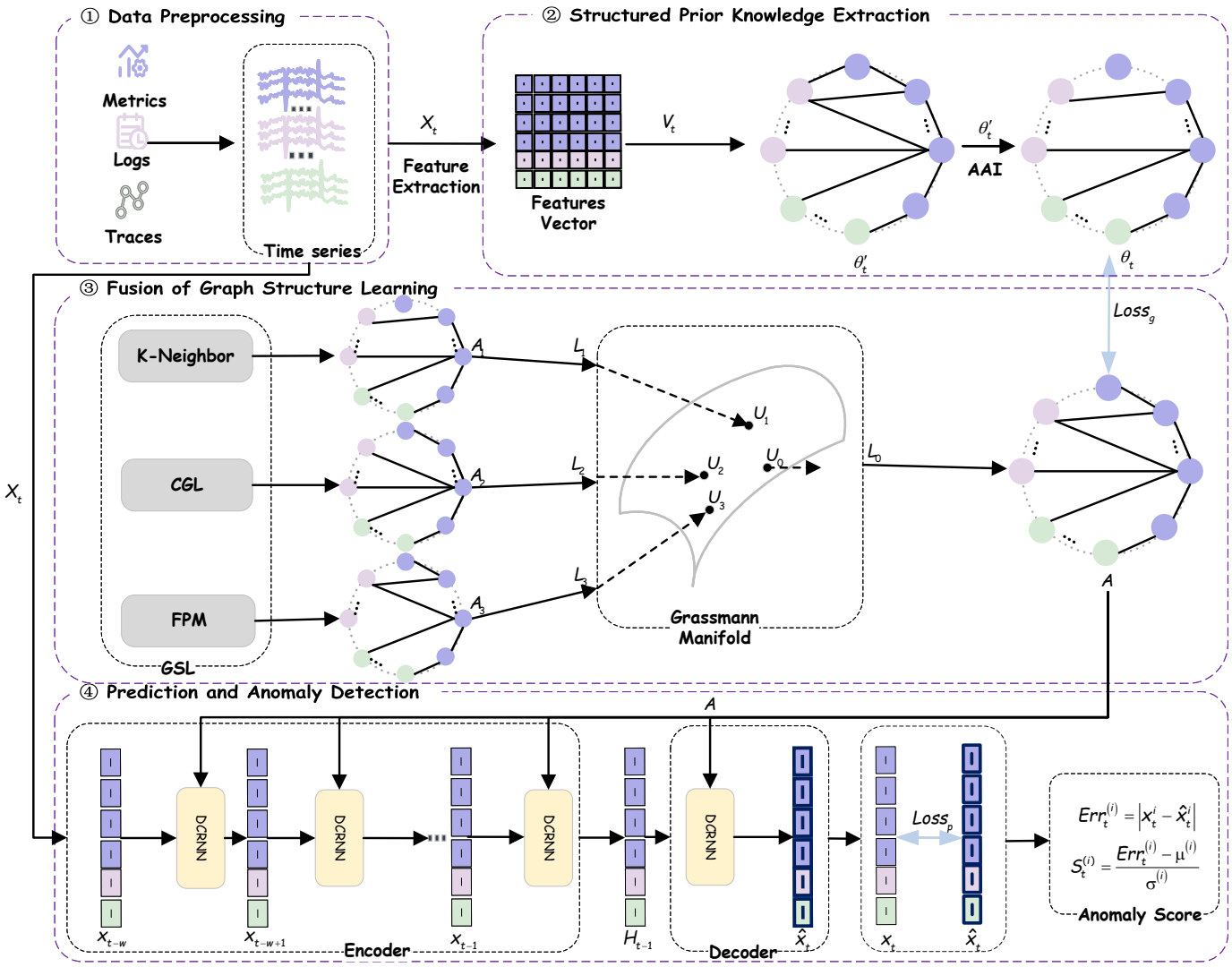


Fig. 4: The framework of MGFusion.

is the anomaly scoring function, $\hat{x}_t \in \mathbb{R}^K$ is the predicted value, and $\hat{y}_t$ is the binary anomaly detection result. An anomaly is detected at timestamp t if $s_t > T$.

C. Solution Overview

In microservice systems, multimodal data often exhibit complex and multi-view relationships that are challenging to model effectively. To address this, we propose **MGFusion**, an efficient framework for multimodal anomaly detection. MGFusion transforms multimodal data into time series representations, incorporates structured prior knowledge to guide graph structure learning, and employs Grassmann manifolds-based graph fusion to combine multiple basic graph structures, enabling the learning of complex and multi-view relationships. The framework of MGFusion is illustrated in Fig. 4, which comprises four main components:

- **Data Preprocessing:** MGFusion employs a robust method to convert all multimodal data into a unified time series format to preserve critical information. This process produces the converted multivariate time series X, derived from the raw metrics $Data_M$, logs $Data_L$, and traces $Data_T$.
- **Structured Prior Knowledge Extraction:** MGFusion leverages the Adamic-Adar Index to evaluate node similarity and extract structured prior knowledge θ_t. This prior knowledge provides guidance for graph structure learning, helping to reduce the impact of noise and enhance the accuracy of relationship modeling.
- **Fusion of Graph Structure Learning:** MGFusion employs three basic GSL methods to generate multiple basic graph structures: A_1, A_2, and A_3. These graphs capture different perspectives of the relationships among the converted multivariate time series. To effectively combine these basic graphs and represent the complex, multi-view relationships, MGFusion uses Grassmann manifolds-based graph fusion. The resulting fused graph structure, denoted as A, encodes the optimal relationships for anomaly detection.
- **Prediction and Anomaly Detection:** MGFusion uses the Diffusion Convolutional Recurrent Neural Network (DCRNN) to predict future values $\hat{x}_t$ of the multivariate

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:31:17 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3665382

time series. Anomaly scores are calculated based on the difference between the predicted values $\hat{x}_t$ and the actual observed values x_t. If the anomaly score exceeds a predefined threshold, the timestamp t is flagged as anomalous.

In the following sections, we provide a detailed explanation of each step in the framework.

IV. Data Preprocessing

The aim of data preprocessing is to robustly convert all multimodal data into a unified time series format. This step ensures that metrics, logs, and traces—each with unique characteristics—are consistently represented, making them suitable for further analysis and anomaly detection.

When converting multimodal data into aligned time series, all raw data must be mapped to values within fixed intervals (e.g., one minute). This process presents a challenge because multiple records may occur within a single interval, and the data volume per interval often varies across modalities. For instance, as shown in Fig. 1, two log messages (L1, L2) and two spans (T1, T2) are generated at 2021-07-01 09:57. The core task of this conversion process is to aggregate multiple records within an interval into a single unified representation. Metrics are typically represented as continuous data, while logs are semi-structured text and discrete in nature. Traces, on the other hand, include both response time (a continuous variable) and status codes (a discrete variable). A general aggregation method is averaging; however, applying naive averaging to discrete data, such as status codes, can lead to meaningless or misleading results.

Therefore, careful consideration of the data type is essential when designing aggregation strategies for multimodal time series conversion. For robust time series conversion, continuous data are aggregated using statistical methods, while discrete data are converted using frequency encoding, which counts the occurrences of each event type per interval. This ensures that meaningful information is preserved.

- **Metric Preprocessing:** Metrics provide valuable insights into the performance of services and machine states (e.g., CPU usage, memory usage, and throughput). In practice, low-variance metrics are filtered out, as they are statistically less likely to contain meaningful anomaly information. This step reduces the dimensionality of the data while retaining critical metrics. After preprocessing, metrics are represented as:

$$X_{\text{Metrics}} = \text{TSM}(Data_M) \in \mathbb{R}^{K_M \times N},$$

where N is the number of timestamps, and K_M is the number of selected metrics.

- **Log Preprocessing:** Logs are semi-structured data that record system status and significant events. To process logs, a log parsing tool, such as Drain [41], is used to convert semi-structured log messages into structured log templates. Each log is mapped to a log template. Frequency encoding is applied to count the occurrences of each template per interval, resulting in a time series representation of logs:

$$X_{\text{Logs}} = \text{TSL}(Data_L) \in \mathbb{R}^{K_L \times N},$$

where K_L is the number of unique log templates.

- **Trace Preprocessing:** Traces capture the execution process of user requests in microservices. They provide response time statistics and feedback on server status. The response time of all traces within each interval is aggregated into four statistical features: mean, median, range, and standard deviation. Status codes in the traces are categorized into five response categories based on their first digit, as detailed in Table II. Unlike AnoFusion [18], which retains only the status code of the first span, our method applies frequency encoding to count the occurrences of each response category (e.g., 2xx, 4xx, 5xx, etc.) per minute. This process generates five additional time series, ensuring that all status codes within an interval are preserved. Traces are represented as:

$$X_{\text{Traces}} = \text{TST}(Data_T) \in \mathbb{R}^{K_T \times N},$$

where $K_T = 9$ (four RT statistics and five response categories).

TABLE II: Status Code Categories.

Status Code Type	Response Category
1xx	Informational
2xx	Success
3xx	Redirection
4xx	Client Error
5xx	Server Error

After obtaining the three types of time series, we synchronize and merge them. Metrics, logs, and traces are normalized and combined into a new multivariate time series $X \in \mathbb{R}^{K \times N}$, where $K = K_M + K_L + K_T$ is the total number of time series. This preprocessing method is designed with a certain level of generality and can be applied to different datasets. Meanwhile, we adapt a flexible strategy to handle different situations of data loss. In practice, for a small amount of missing data or when the missing data is independent, we use data imputation methods to recover the missing values. However, in cases of continuous and extensive data loss (e.g., when data for an entire day is missing), we remove the corresponding records to ensure data quality.

Taking the data in Fig. 1 as an example, Fig. 5 illustrates serialization and graph construction.

- **Logs:** Two log messages at 2021-07-01 09:57 are parsed into Template1 and Template2, resulting in the converted log time series for 09:57 as $[1, 1, 0, \ldots, 0]^{\text{T}}$. At 09:58, one log message is parsed into Template2, resulting in $[0, 1, 0, \ldots, 0]^{\text{T}}$.

- **Traces:** For spans T1 and T2, the RT statistics are calculated as mean = 12.5, median = 12.5, range = 5, and standard deviation = 2.5. Status codes are categorized into 2XX and 5XX. For span T3 and T4, the RT statistics are calculated as mean = 10, median = 10, range = 0, and standard deviation = 0, with the status code categorized into 2XX. The converted trace time series for 09:57 is $[12.5, 12.5, 5, 2.5, 0, 1, 0, 0, 1]^{\text{T}}$, and for 09:58 is $[20, 20, 0, 0, 0, 2, 0, 0, 0]^{\text{T}}$. As a result, both the 2XX and 5XX status codes are preserved, effectively addressing the issue outlined in the first challenge.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:31:17 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3665382

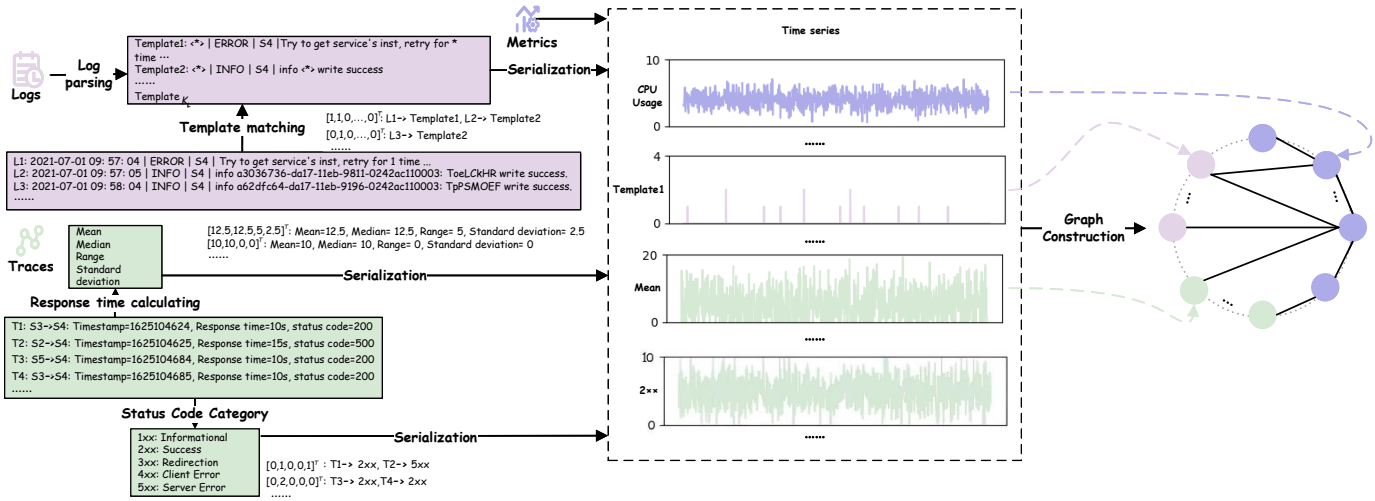


Fig. 5: Serialization of multimodal data and graph construction.

Treating each time series as a node, the converted multivariate time series can be represented as the graph $G = (V, E)$ where the relationships between the nodes (time series) are initially unknown. To uncover these hidden relationships, graph structure learning is employed to dynamically learn the optimal adjacency matrix.

V. Structural Prior Knowledge Extraction

To guide graph structure learning, we incorporate prior knowledge, typically represented as a prior graph derived from raw data. This prior graph is generally constructed based on node feature similarity. However, in real-world scenarios, data noise can distort node feature representations, leading to spurious or noisy edges in the graph. These artifacts degrade the quality of the learned graph structure and negatively impact the performance of downstream anomaly detection task.

To mitigate the impact of noisy edges, we propose a structural prior graph generation method based on the **Adamic-Adar Index (AAI)**. AAI accounts for the importance of neighbors, effectively capturing true relationships between nodes while reducing the influence of data noise. The process consists of two stages: constructing an initial graph and refining it using the AAI to create a structured prior graph.

A. Initial Prior Graph Construction

We start by constructing an initial prior graph using a similarity-thresholding approach [42]. Node features are extracted using two one-dimensional convolutional layers (Conv) followed by a fully connected layer (FC), as shown in Eq. (5). Cosine similarity is calculated between feature vectors of nodes, as shown in Eq. (6). Nodes with similarity scores above a predefined threshold τ are treated as neighbors, forming the initial prior graph θ'_t.

$$V_t^{(i)} = \mathrm{FC}\left(\mathrm{Conv}\left(\mathrm{Conv}\left(x_t^{(i)}\right)\right)\right), \tag{5}$$

$$\cos(V_t^{(i)}, V_t^{(j)}) = \frac{V_t^{(i)} \cdot V_t^{(j)}}{\|V_t^{(i)}\| \cdot \|V_t^{(j)}\|}, \tag{6}$$

where $x_t^{(i)} = [x_{t-w}^i, x_{t-w+1}^i, \ldots, x_{t-1}^i] \in \mathbb{R}^w$ is the historical subsequence of node i at timestamp t, $V_t^{(i)}$ is the extracted feature vector for node i, and $\|\cdot\|$ represents the vector norm. The set of neighbors for each node i is denoted by $\Gamma_t^{(i)}$.

$$\Gamma_t^{(i)} = \{j \mid \cos(V_t^{(i)}, V_t^{(j)}) > \tau\}. \tag{7}$$

B. Adamic-Adar Index-Based Refinement

The AAI is a node-local similarity metric that emphasizes the contribution of low-degree shared neighbors, as these are more informative in similarity measurement than high-degree ones. The AAI between nodes i and j is defined as:

$$AAI^{(i,j)} = \sum_{u \in \Gamma^{(i)} \cap \Gamma^{(j)}} \frac{1}{\ln |\Gamma^{(u)}|}, \tag{8}$$

where $\Gamma^{(i)}$ and $\Gamma^{(j)}$ are the neighbor sets of nodes i and j, and $|\Gamma^{(u)}|$ is the degree of neighbor u.

AAI assigns higher importance to neighbors of low degrees, reducing the influence of noisy, high-degree nodes. Using AAI, we refine the initial prior graph as follows. We calculate the AAI for all pairs of nodes in the initial prior graph. Each node i select the k nodes with the highest AAI scores as its neighbors. The refined structural prior graph is represented by the adjacency matrix θ_t.

$$\theta_t^{(i,j)} = 1, \quad \text{if } j \in \text{top_k}\left(AAI^{(i,:)}\right), \tag{9}$$

where top_k selects the indices of the k nodes with the highest AAI scores.

By prioritizing low-degree neighbors, AAI suppresses the impact of noisy or spurious edges caused by data noise. The graph structure learned using this prior knowledge is more robust and adaptable to the requirements of downstream anomaly detection task.

VI. Fusion of Graph Structure Learning

To represent the complex relationships between converted time series from multiple views, we first employ three basic graph structure learning methods to learn the relationships.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:31:17 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3665382

These methods capture different perspectives of node interactions. Then, we use Grassmann manifolds-based graph fusion to combine the learned graphs into a unified structure.

A. Basic Graph Structure Learning

To enhance the quality of graph structure learning, we use three representative GSL methods: K-Nearest Neighbors, Causal Graph Learning, and Full Parameterization Method. Each method captures node interactions from a distinct perspective, providing a comprehensive understanding of relationships such as similarity and causality. Together, these methods provide complementary insights into the structure of the graph, enabling the construction of a high-quality adjacency matrix that reflects both direct and indirect interactions among nodes.

- **K-Nearest Neighbors Method (K-Neighbor):** This method constructs edges based on cosine similarity between node embeddings, constructing edges between nodes with the closest relationships. For each node i, a learnable embedding vector $E^{(i)}$ is generated. The k-nearest neighbors are selected based on cosine similarity to construct the adjacency matrix A_1:

$$S_1^{(i,j)} = \cos(E_t^{(i)}, E_t^{(j)}) = \frac{E_t^{(i)} \cdot E_t^{(j)}}{\|E_t^{(i)}\| \cdot \|E_t^{(j)}\|}, \tag{10}$$

$$A_1^{(i,j)} = \begin{cases} 1, & \text{if } j \in \text{top_k}\left(S_1^{(i,:)}\right) \\ 0, & \text{otherwise.} \end{cases}, \tag{11}$$

  Here, top_k selects the indices of the k nodes with the highest similarity.
- **Causal Graph Learning (CGL):** This method captures causal impacts between nodes by constructing an asymmetric adjacency matrix, capturing the influence of one node on another. The Source and destination embeddings (E_1 and E_2) are learned for each node. The adjacency matrix A_2 is derived as:

$$M_1 = \tanh(\phi E_1 \beta_1), \tag{12}$$

$$M_2 = \tanh(\phi E_2 \beta_2), \tag{13}$$

$$S_2 = \text{ReLU}(\tanh(\phi(M_1 M_2^T - M_2 M_1^T))), \tag{14}$$

$$A_2^{(i,j)} = \begin{cases} 1, & \text{if } j \in \text{top_k}(S_2^{(i,:)}) \\ 0, & \text{otherwise.} \end{cases}, \tag{15}$$

  where the asymmetric structure captures directionality. The top k largest values in $A_2^{(i,:)}$ are retained as edges.
- **Full Parameterization Method (FPM):** This method uses Gumbel Softmax to directly learn the graph structure. A probability matrix $\pi_1 \in \mathbb{R}^{K \times K}$ determines edge existence:

$$g^{(i,j)} = -\log(-\log(u)),\ u \sim \text{Uniform}(0,1), \tag{16}$$

$$A_3^{(i,j)} = \frac{\exp\left(\left(\log \pi_1^{(i,j)} + g^{(i,j)}\right)/\tau\right)}{\sum_{v\in\{0,1\}} \exp\left(\left(\log \pi_v^{(i,j)} + g^{(i,j)}\right)/\tau\right)}, \tag{17}$$

  where u is a random sample drawn from the Uniform$(0,1)$ distribution, and $g^{(i,j)}$ follows the Gumbel distribution. The parameter τ is the temperature parameter, controlling the sharpness of the Gumbel-Softmax distribution. As τ approaches 0, $A_3^{(i,j)}$ converges to either 0 or 1, making the Gumbel-Softmax distribution behave like a categorical distribution. The adjacency matrix element $A_3^{(i,j)}$ is 1 with probability $\pi_1^{(i,j)}$, reflecting the likelihood of a connection between nodes i and j.

B. Grassmann Manifolds-Based Graph Fusion

After generating the three graphs (A_1, A_2, A_3), the critical challenge lies in effectively combining them. The combining methods include averaging and weighted averaging. While simple averaging offers a straightforward solution, it fails to account for the relative importance of each graph structure. Alternatively, similarity-weighted averaging risks distorting the original structural properties of the constituent graphs. Grassmann manifolds-based subspace fusion can merge multiple subspaces or graphs into a representative subspace while preserving the geometric features of each subspace or graph [43]. Therefore, we combine them using Grassmann manifolds to preserve their geometric properties.

*1) **Grassmann Manifold**:* Grassmann manifold is a mathematical framework about the geometric property of linear subspaces. It views all linear subspaces of the same dimension as a manifold, represented by $Gr(n,p)$. It includes all n-dimensional linear subspaces in a p-dimensional Euclidean space, denoted by the set of orthogonal matrices Y.

$$Gr(n,p) = \{Y | Y \in R^{n\times p}, Y^{\mathrm{T}} Y = I\}. \tag{18}$$

For an adjacency matrix A_i of i-th basic graph, its Laplacian matrix L_i has a complete set of orthonormal eigenvectors, denoted by eigenvector matrices U_i:

$$\begin{aligned} L_i &= D_i^{-\frac{1}{2}}(D_i - A_i)D_i^{-\frac{1}{2}}, \\ L_i &= U_i \Lambda_i U_i^{\mathrm{T}}, \quad U_i^{\mathrm{T}} U_i = I, \end{aligned} \tag{19}$$

where $D_i \in R^{K\times K}$ is the degree matrix of A_i, $L_i \in R^{K\times K}$ is the symmetrically normalized Laplacian matrix of A_i, $U_i \in R^{K\times p}$ is the eigenvector matrix, and $\Lambda_i \in R^{p\times p}$ is the eigenvalue diagonal matrix.

U_i lies on the Grassmann manifold $Gr(K,p)$:

$$U_i \in Gr(K,p). \tag{20}$$

The three graphs are mapped to the Grassmann manifold through their eigenvector matrices U_1, U_2, U_3.

*2) **Subspace Fusion on Grassmann Manifolds**:* The goal of subspace fusion is to find a representative subspace U_0 that: (1) Integrates information from U_1, U_2, U_3, (2) Preserves the geometric structure of each subspace. Therefore, two objectives need to be minimized.

The first objective is to preserve the geometric features of each subspace. It can be achieved by minimizing the Laplacian quadratic form $\sum_{i=1}^{3} \text{tr}(U^T L_i U)$.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:31:17 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3665382

The second objective is to minimize the projection distance between the representative subspace and each subspace U_i:

$$\begin{aligned}
&\min_{U\in R^{K\times p}} \sum_{i=1}^{3} d^2(U,U_i) = \min_{U\in R^{K\times p}} \sum_{i=1}^{3}\sum_{j=1}^{p} \sin^2\theta_{ij} \\
&= \min_{U\in R^{K\times p}} \sum_{i=1}^{3} \left(p - \sum_{j=1}^{p} \cos^2\theta_{ij} \right) \\
&= \min_{U\in R^{K\times p}} \sum_{i=1}^{3} \left(p - \mathrm{tr}(UU^T U_i U_i^T) \right) \\
&= \min_{U\in R^{K\times p}} 3p - \sum_{i=1}^{3} \mathrm{tr}(UU^T U_i U_i^T).
\end{aligned} \tag{21}$$

Finally, the optimization problem is as follows:

$$\begin{aligned}
&\min_{U\in\mathbb{R}^{K\times p}} \sum_{i=1}^{3} \mathrm{tr}(U^T L_i U) + \alpha \left(3p - \sum_{i=1}^{3} \mathrm{tr}(UU^T U_i U_i^T) \right), \\
&\text{s.t.} \quad U^T U = I,
\end{aligned} \tag{22}$$

where the first term preserves geometric properties and the second term minimizes projection distances between U_0 and U_i. The parameter α controls the balance between these two terms.

Applying the Rayleigh-Ritz theorem to solve the optimization problem (22), the Laplacian matrix L_0 for the fused graph is:

$$L_0 = \sum_{i=1}^{3} L_i - \alpha \sum_{i=1}^{3} U_i U_i^T. \tag{23}$$

Finally, the adjacency matrix A_0 is derived as follows:

$$A_0 = D_0 - L_0. \tag{24}$$

*3) **Graph Sparsification and Finalization**:* The adjacency matrix of the combined graph may contain negative values, whereas real-world graphs typically have non-negative adjacency matrices. Therefore, we apply a piecewise linear unit function to eliminate negative values.

To reduce computational complexity, we sparsify A_0 by retaining only the top k edges for each node:

$$\hat{A}^{(i,j)} = \begin{cases} A_0^{(i,j)}, & \text{if } A_0^{(i,j)} \in \text{top_k}(A_0), \\ 0, & \text{otherwise.} \end{cases} \tag{25}$$

The final symmetric adjacency matrix is:

$$A = \frac{\hat{A} + \hat{A}^T}{2}. \tag{26}$$

This Grassmann manifolds-based graph fusion effectively integrates multiple graph structures while preserving their unique properties, enabling robust graph representation.

VII. DCRNN Predictor and Anomaly Detection

Based on the combined graph, MGFusion predicts future values using a spatio-temporal GNN and detects anomalies by analyzing the differences between predictions and actual values.

A. DCRNN Predictor

The DCRNN is a powerful spatio-temporal GNN that effectively captures both spatial and temporal dependencies. Compared to conventional Graph Convolutional Networks, DCRNN is more suitable for time series prediction due to its ability to model complex interactions over both space and time. DCRNN integrates two key components: diffusion convolution for capturing spatial relationships and Gated Recurrent Unit (GRU) for modeling temporal dynamics.

Diffusion Convolution: The diffusion convolution mechanism aggregates L-hop neighbor features and is defined as:

$$W_Q \circ Y = \sum_{i=0}^{L} \left(w_{i,1}^{Q} \left(D_O^{-1} A \right)^i + w_{i,2}^{Q} \left(D_I^{-1} A^T \right)^i \right) Y, \tag{27}$$

where $\circ$ represents the diffusion convolution operation, A is the combined adjacency matrix, D_O and D_I are the out-degree and in-degree matrices, respectively, $w_{i,1}^{Q}$, $w_{i,2}^{Q}$, and b_Q are learnable parameters, and L is the diffusion degree (a hyperparameter).

This mechanism allows DCRNN to aggregate multi-hop spatial dependencies while preserving the directionality of the graph.

Diffusion Convolutional Recurrent Layer: The diffusion convolutional recurrent layer combines diffusion convolution with GRU to capture spatio-temporal dynamics. The equations governing the layer are as follows:

$$R_t = \text{sigmoid}\left(W_R \circ [x_t \parallel H_{t-1}] + b_R \right), \tag{28}$$

$$C_t = \tanh\left(W_C \circ [x_t \parallel (R_t \circ H_{t-1})] + b_C \right), \tag{29}$$

$$U_t = \text{sigmoid}\left(W_U \circ [x_t \parallel H_{t-1}] + b_U \right), \tag{30}$$

$$H_t = U_t \circ H_{t-1} + (1 - U_t) \circ C_t, \tag{31}$$

where $\parallel$ denotes concatenation, $x_t \in \mathbb{R}^K$ represents the input values at timestamp t, and H_t represents the node features at timestamp t.

The diffusion convolutional recurrent layer takes x_t and the previous node features H_{t-1} as inputs and outputs the updated node features H_t.

DCRNN Prediction Process: DCRNN employs an encoder-decoder architecture for time series prediction. The prediction process comprises the following steps. The encoder processes the input sequence x_t from timestamp $t-\omega$ to $t-1$. It sequentially updates the node features for each timestamp, resulting in the aggregated node features H_{t-1}. The decoder takes the encoded node features H_{t-1} as input. Using a diffusion convolutional recurrent layer, the decoder predicts the future values $\hat{x}_t$ at timestamp t.

By combining diffusion convolution for spatial dependencies and GRU for temporal dynamics, DCRNN provides a highly effective framework for spatio-temporal modeling and accurate time series prediction.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:31:17 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3665382

B. Loss Function

The model's objective is to minimize the prediction error while ensuring the quality of the learned graph structure. The total loss function consists of three components:

Prediction Loss:The mean absolute error (MAE) is used as prediction loss:

$$loss_p = \frac{1}{K}\sum_{i=1}^{K}\left|\hat{x}_t^i - x_t^i\right|, \tag{32}$$

where $\hat{x}_t^i$ and x_t^i are the predicted and actual values for the i-th time series, respectively.

Graph Structure Learning Loss: To improve graph quality, the structured prior graph θ_t is used as prior knowledge. The GSL loss [44] is defined as the cross-entropy between θ_t and the learned graph A:

$$loss_g = \sum_{ij} -\theta_t^{(i,j)} \log A^{(i,j)} - (1-\theta_t^{(i,j)}) \log(1-A^{(i,j)}). \tag{33}$$

L_2 Regularization: To reduce overfitting, an L_2 regularization term is added:

$$\lambda_2 \|w\|_2^2, \tag{34}$$

where w represents the model parameters.

The total loss function is as follows:

$$loss = loss_p + \lambda_1 loss_g + \lambda_2 \|w\|_2^2, \tag{35}$$

where λ_1 and λ_2 are regularization coefficients.

C. Anomaly Score

To detect anomalies, we calculate an anomaly score based on the deviation of predicted values from actual values.

Prediction Error: The prediction error for the i-th time series at timestamp t is:

$$Err_t^{(i)} = \left|x_t^i - \hat{x}_t^i\right|. \tag{36}$$

Standardization: To ensure consistency in different time series, the prediction error is standardized.

$$s_t^{(i)} = \frac{Err_t^{(i)} - \mu^{(i)}}{\sigma^{(i)}}, \tag{37}$$

where $\mu^{(i)}$ and $\sigma^{(i)}$ are the mean and standard deviation of $Err_t^{(i)}$, respectively.

Anomaly Score: The maximum standardized prediction error across all time series is the anomaly score for timestamp t:

$$s_t = \max_i s_t^{(i)} = \max_i \frac{Err_t^{(i)} - \mu^{(i)}}{\sigma^{(i)}}. \tag{38}$$

Threshold Selection: The optimal threshold for anomaly detection is determined using grid search. The search range is defined by the minimum and maximum values of s_t. The threshold that achieves the highest F1 score is selected. To improve robustness, a point adjustment strategy is applied to smooth the anomaly detection results.

With this procedure, MGFusion effectively detects anomalies based on accurate predictions.

In the Artificial Intelligence for IT Operations (AIOps) field, the MGFusion framework can serve as a core component of intelligent operation and maintenance platforms for real-time monitoring and anomaly detection. By MGFusion, operation and maintenance teams can achieve automatic anomaly detection.

VIII. Experiments

In this section, we perform evaluation experiments to address the following research questions:

- **RQ1 (Detection Performance and Efficiency):** Is our method superior to baseline methods in terms of anomaly detection performance and efficiency?
- **RQ2 (Ablation Study):** How does each component of MGFusion contribute to its overall performance?
- **RQ3 (Parameter Sensitivity):** How sensitive is MGFusion to variations in key parameters?

A. Experimental Setup

1) Dataset: We conduct experiments using the General AIOps Atlas (GAIA) dataset[1] provided by CloudWise. GAIA is a comprehensive dataset designed for AIOps research, covering tasks such as anomaly detection, log analysis, and anomaly localization. The dataset simulates a system consisting of five hosts serving both PC and mobile users simultaneously. It includes five microservices deployed across ten instances, with operators injecting five types of anomalies: system stuck, process crash, log-in failure, file missing, and access denied.

Each microservice maintains two separate instances (e.g., 'Webservice1' and 'Webservice2'). Despite providing the same service functionality, these instances exhibit significant operational variations, such as differences in log templates, data volumes, and metric variations. To account for these differences, we perform instance-level anomaly detection and evaluate two configurations of the dataset:

- **D1**: Includes three services: 'Mobservice', 'Loginservice', and 'Redisservice'.
- **D2**: Covers all services (D1 + 'Dbservice' + 'Webservice').

For the experiments, we analyze half a month of data. The detailed dataset statistics are presented in Table III.

TABLE III: Dataset statistics.

Dataset	Metrics	Logs	Traces	Anomaly Ratio
$\mathcal{D}1$	423,359	51,431,324	19,437,955	8.84%
$\mathcal{D}2$	734,165	87,974,577	28,681,438	8.41%

2) Baselines: We compare MGFusion with ten state-of-the-art methods, including single-modal and multimodal approaches. The single-modal anomaly detection methods include OmniAnomaly [45], AnomalyTrans [46], Deeplog [32], LogRobust [35], and SwissLog [47]. The multimodal anomaly detection methods with two modalities, include MADMM

[1]https://github.com/CloudWise-OpenSource/GAIA-DataSet

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:31:17 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3665382

[13], SCWarn [21], and Hades [40]. The multimodal anomaly detection methods with three modalities are MSTGAD [19] and AnoFusion [18].

While there is a significant body of excellent work in the field of multimodal anomaly detection, such as Nezha, Eadro, and MRCA [20], a direct comparison with these methods is unfortunately not feasible due to differences in task objectives. Nezha, Eadro, and MRCA aim to detect whether an anomaly has occurred in the entire system and, if so, locate the root microservice causing the issue. However, these methods do not address the identification of which specific microservices exhibit anomalous behavior.

3) Experimental Environment and Setup: The experimental parameter configurations are detailed in Table IV. MGFusion is implemented using PyTorch. All experiments are conducted in Python 3.8 and PyTorch 1.10, with CUDA 11.3 for GPU acceleration. The hardware environment includes an Intel(R) Core(TM) i7-13700KF CPU and an NVIDIA RTX 4060 GPU.

TABLE IV: Method Parameters.

Parameter	Value
Window Size ω	20
Hyperparameter α	0.7
Graph Loss Regularization Parameter λ_1	1
Learning Rate	0.005
Batch Size	64
Epochs	30
k in K-nearest Neighbors Graph Structure Learning	40
Diffusion Degree L	5

The multimodal dataset is divided into training and testing sets. The training set contains the first 70% of the data from each instance. The testing set contains the remaining 30%.

4) Evaluation Metrics: We use standard metrics for anomaly detection, including precision ($Prec$), recall (Rec), and F1 score ($F1$), defined as follows:

$$Prec = \frac{TP}{TP + FP}, \tag{39}$$

$$Rec = \frac{TP}{TP + FN}, \tag{40}$$

$$F1 = 2 \times \frac{Prec \times Rec}{Prec + Rec}, \tag{41}$$

where TP, FP and FN represent the number of true positives, false positives, and false negatives, respectively.

We also consider the training time and interfere time as metrics to evaluate the efficiency.

B. Detection Performance and Efficiency (RQ1)

1) Detection Performance: In this experiment, we compare the performance of MGFusion against all baseline methods. Specifically, we evaluate precision, recall, and F1 score across different datasets. However, since MADMM is not an open-source method, it is not feasible to apply MADMM to the D2 dataset and obtain the necessary performance metrics for comparison. MSTGAD focuses on anomaly detection across all service instances in a system, requiring traces from all service instances. Consequently, it can only support the D2 dataset, which contains all instances from the GAIA dataset. This approach results in extremely large multimodal datasets and takes over 7 hours to process the GAIA dataset, even when limited to the first 20,000 data points (seconds) with a training-to-testing ratio of 7:3.

As shown in Table V, MGFusion outperforms all baseline methods on both datasets, achieving the best F1 scores of 0.958 on D1 and 0.968 on D2. Single-modal anomaly detection methods, such as OmniAnomaly and AnomalyTrans, demonstrate limited performance due to their inability to leverage multimodal information. In contrast, dual-modal methods significantly improve detection accuracy by incorporating relationships between metrics and logs. While supervised and semi-supervised methods generally achieve higher performance, MGFusion, despite being fully unsupervised, matches or surpasses the F1 scores of top supervised methods by leveraging the benefits of trace data. Trace data captures important contextual and temporal information that enhances the model's ability to detect anomalies. Furthermore, it outperforms all existing unsupervised methods, highlighting its exceptional capability to detect anomalies in unlabeled datasets.

As shown in Fig. 6, the ROC curves for the two datasets (D1 and D2) exhibit AUC of 0.976 for D1 and 0.977 for D2. These results further highlight the effectiveness and superiority of our method in anomaly detection.

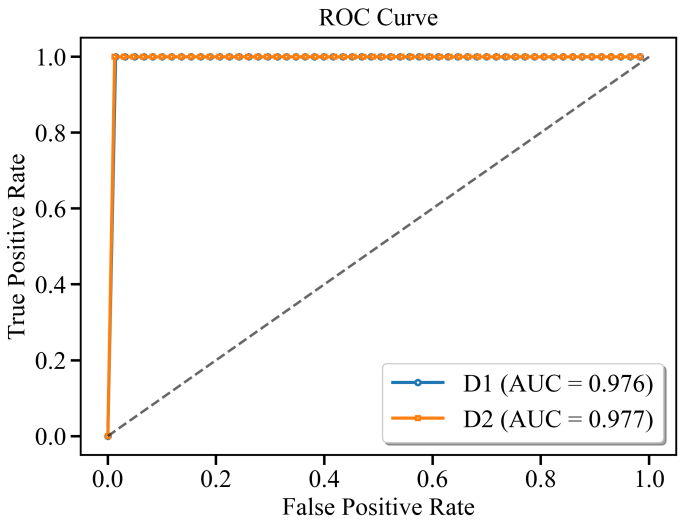


Fig. 6: The ROC-AUC curve of the model.

2) Detection Efficiency: We further evaluate the efficiency of MGFusion by comparing its training and inference times with AnoFusion [18] on both datasets. As shown in Table VI, MGFusion demonstrates significantly shorter training and inference time.

AnoFusion relies on complex multi-layer graph transformation networks and graph attention networks to model relationships between multimodal data, resulting in higher computational complexity. In contrast, MGFusion leverages DCRNN for prediction, which reduces training time while maintaining high performance. Additionally, MGFusion's faster inference time makes it more suitable for real-time systems.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:31:17 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3665382

TABLE V: Performance comparison among different methods. The highest and second-highest results are highlighted with boldface and underline.

Method	Type	Modality			$\mathcal{D}1$			$\mathcal{D}2$		
		Metric	*Log*	*Trace*	*Prec*	*Rec*	*F1*	*Prec*	*Rec*	*F1*
OmniAnomaly	Unsupervised	✓			0.358	0.510	0.420	0.373	0.563	0.448
AnomalyTrans	Unsupervised	✓			0.452	0.673	0.541	0.453	0.688	0.546
Deeplog	Semi-supervised		✓		0.564	0.886	0.689	0.506	0.812	0.623
LogRobust	Supervised		✓		0.692	0.744	0.719	0.710	0.772	0.740
SwissLog	Supervised		✓		0.747	0.775	0.761	0.759	0.812	0.784
MADMM	Supervised	✓	✓		<u>0.937</u>	<u>0.981</u>	**0.958**	–	–	–
SCWarn	Unsupervised	✓	✓		0.499	0.642	0.561	0.547	0.425	0.447
Hades	Semi-supervised	✓	✓		**0.940**	0.962	<u>0.950</u>	<u>0.936</u>	<u>0.959</u>	<u>0.947</u>
MSTGAD	Semi-supervised	✓	✓	✓	–	–	–	<u>0.936</u>	0.898	0.917
AnoFusion	Unsupervised	✓	✓	✓	0.804	0.864	0.805	0.795	0.945	0.857
MGFusion	Unsupervised	✓	✓	✓	0.926	**0.998**	**0.958**	**0.943**	**0.998**	**0.968**

TABLE VI: Comparison of detection efficiencies.

Dataset	Method	Training Time (s/epoch)	Inference Time (s)
$\mathcal{D}1$	AnoFusion	12.47s	74.76s
	MGFusion	5.91s	45.12s
$\mathcal{D}2$	AnoFusion	12.82s	80.84s
	MGFusion	6.71s	47.51s

In summary, MGFusion achieves state-of-the-art detection performance while significantly improving computational efficiency, making it a highly effective solution for anomaly detection in real-world systems.

C. Ablation Experiment (RQ2)

To demonstrate the contribution and importance of each component of MGFusion, we create four variants of MGFusion and conduct a series of experiments to compare their performance. The variants are defined as follows:

- **MGFusion w/o Graph Fusion.** To evaluate the importance of the multiple graphs and graph fusion, this variant uses only the FPM in place of the three graph structure learners.
- **MGFusion w/o Grassmann.** This variant removes the Grassmann manifolds-based graph fusion and instead combines graphs based on their similarity to the prior graph. This evaluates the impact of Grassmann manifolds-based graph fusion.
- **MGFusion w/o Prior Knowledge.** To examine the role of prior knowledge in guiding GSL, this variant eliminates the GSL loss, effectively removing the influence of prior knowledge.
- **MGFusion w/o DCRNN.** To evaluate whether DCRNN is the optimal choice, we replace DCRNN with Spatial-Temporal Graph Convolutional Networks (STGCN) [48].

Table VII reports the average precision, recall, and F1 scores of the four variants. The results indicate the following.

The three GSL methods provide complementary perspectives for capturing complex node relationships, which cannot be fully replicated by a single GSL method (e.g., FPM). Consequently, the performance of MGFusion significantly degrades when Graph Fusion is removed (MGFusion w/o Graph Fusion).

The Grassmann manifolds-based graph fusion module is critical, as it preserves the distinct structural properties of basic graphs while constructing a unified representation. Removing this component leads to a notable performance drop, underscoring its necessity.

The prior graph serves as a structural guide for graph learning. Removing prior knowledge (MGFusion w/o Prior Knowledge) results in a decline in precision, highlighting the importance of this inductive bias for robust anomaly detection.

The DCRNN plays a crucial role in the MGFusion. In Table VII, when the DCRNN is replaced by STGCN (MGFusion w/o DCRNN), the model's performance significantly decreases, especially in terms of precision.

TABLE VII: Performance comparison among different variants.

Dataset	Method	*Prec*	*Rec*	*F1*
$\mathcal{D}1$	MGFusion w/o Graph Fusion	0.858	0.977	0.909
	MGFusion w/o Grassmann	<u>0.880</u>	0.980	<u>0.924</u>
	MGFusion w/o Prior Knowledge	0.862	<u>0.994</u>	0.920
	MGFusion w/o DCRNN	0.849	0.920	0.873
	MGFusion	**0.926**	**0.998**	**0.958**
$\mathcal{D}2$	MGFusion w/o Graph Fusion	0.888	0.985	0.931
	MGFusion w/o Grassmann	<u>0.905</u>	0.987	<u>0.941</u>
	MGFusion w/o Prior Knowledge	0.893	<u>0.994</u>	0.938
	MGFusion w/o DCRNN	0.854	0.951	0.892
	MGFusion	**0.943**	**0.998**	**0.968**

D. Parameter Sensitivity (RQ3)

To evaluate the stability and robustness of MGFusion, we analyze the influence of key hyperparameters.

1) Window Size ω: We investigate the impact of the sliding window size by varying it from 5 to 60. Figure 7 shows the experimental results, which highlight the following trends.

Excessively large windows (e.g., > 35) introduce too much seasonal variation, making it challenging to reconstruct the current state and degrading anomaly detection performance.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:31:17 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3665382

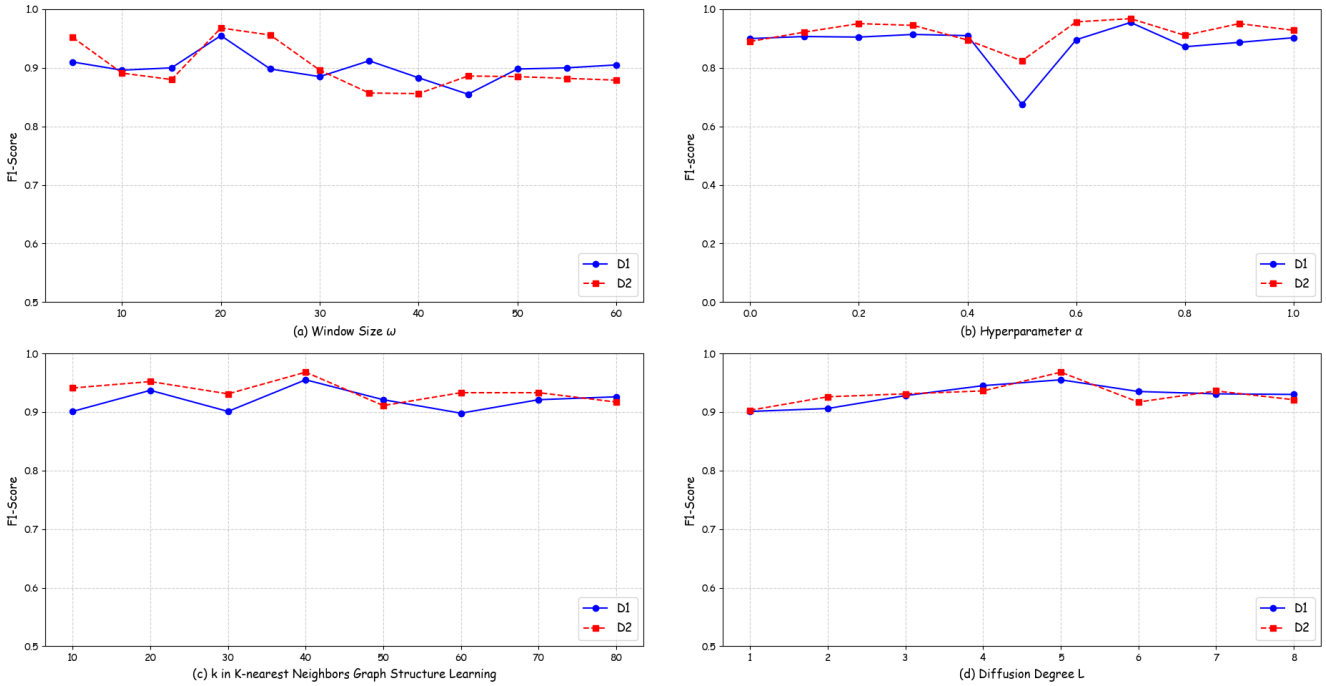


Fig. 7: F1 scores of MGFusion under different parameter.

Overly small windows (e.g., < 15) fail to capture sufficient historical context, leading to suboptimal learning and reduced detection accuracy. Optimal performance is achieved with a window size between 15 and 35, where a balance is struck between capturing historical patterns and avoiding excessive noise.

2) Hyperparameter α: The Grassmann manifolds-based graph fusion framework uses two key components: the contribution of basic graph structures and the constraints of common structures. The hyperparameter α balances these components during the fusion process. When $\alpha = 0$, the fusion process considers only the basic graph structures, ignoring subspace consistency constraints. As α increases, the framework emphasizes the global structural information encoded in common structures, progressively aligning subspaces.

We evaluate the impact of α by varying its value from 0 to 1 in steps of 0.1. Figure 7 shows the results. When $\alpha = 0.5$, the information complementarity between basic graph structures and common structures is insufficient, leading to suboptimal fusion performance. Increasing α to 0.7 strengthens subspace consistency constraints, enhancing performance by aligning the global structures more effectively.

These results highlight the importance of balancing local and global structural information for optimal graph fusion.

3) k in K-nearest Neighbors Graph Structure Learning: In K-nearest neighbors graph structure learning, the value of K determines the number of nearest neighbors to consider when constructing the graph. Selecting an appropriate K value is therefore critical for achieving optimal performance. To verify this, we conducted experiments by varying K within a range from 10 to 80. The results showed that the model achieved the best performance when $K = 40$. This highlights the importance of tuning K to ensure the graph accurately captures the relationships among data points.

4) Diffusion Degree L: In our analysis of the diffusion degree L parameter in DCRNN, we systematically varied L from 1 to 8 to evaluate its impact on model performance. The results demonstrated that the model performed optimally when $L = 4$, striking a balance between capturing complex temporal dependencies and mitigating the risk of overfitting. This finding underscores the importance of carefully selecting the diffusion degree L to fully leverage DCRNN's strengths in anomaly detection tasks.

IX. Conclusion

To fully exploit the potential of multimodal data in microservice systems, we propose MGFusion, an innovative, unsupervised, end-to-end multimodal anomaly detection method. MGFusion effectively transforms metrics, logs, and traces into time series using a robust time series conversion approach, enabling consistent processing across all modalities. It leverages GSL methods to capture the intricate relationships between the converted multivariate time series. Additionally, MGFusion incorporates prior knowledge through AAI and employs Grassmann manifolds-based graph fusion to enhance the accuracy and reliability of the learned graph structures. By integrating DCRNN predictors and an anomaly score calculation mechanism, MGFusion achieves efficient and accurate anomaly detection, thereby ensuring service reliability in complex microservice systems. In future, we aim to explore dynamic fusion techniques to better handle the evolving relationships between different modalities.

References

[1] S. Li, L. D. Xu, and S. Zhao, "5g internet of things: A survey," *J. Ind. Inf. Integr.*, vol. 10, pp. 1–9, 2018.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:31:17 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Services Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSC.2026.3665382

[2] Q. Lin, H. Zhang, J. Lou, Y. Zhang, and X. Chen, "Log clustering based problem identification for online service systems," in *ICSE*, 2016, pp. 102–111.

[3] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, and D. Ding, "Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study," *IEEE Trans. Software Eng.*, vol. 47, no. 2, pp. 243–260, 2021.

[4] R. Picoreti, A. P. do Carmo, F. M. de Queiroz, A. S. Garcia, R. F. Vassallo, and D. Simeonidou, "Multilevel observability in cloud orchestration," in *DASC/PiCom/DataCom/CyberSciTech*, 2018, pp. 776–784.

[5] Y. Fu, M. Yan, J. Xu, J. Li, Z. Liu, X. Zhang, and D. Yang, "Investigating and improving log parsing in practice," in *ESEC/FSE*, 2022, pp. 1566–1577.

[6] S. Nedelkoski, J. Cardoso, and O. Kao, "Anomaly detection from system tracing data using multimodal deep learning," in *CLOUD*, 2019, pp. 179–186.

[7] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "USAD: unsupervised anomaly detection on multivariate time series," in *KDD*, 2020, pp. 3395–3404.

[8] M. Ma, S. Zhang, J. Chen, J. Xu, H. Li, Y. Lin, X. Nie, B. Zhou, Y. Wang, and D. Pei, "Jump-starting multivariate time series anomaly detection for online service systems," in *USENIX ATC*, 2021, pp. 413–426.

[9] M. Du, F. Li, G. Zheng, and V. Srikumar, "Deeplog: Anomaly detection and diagnosis from system logs through deep learning," in *CCS*, 2017, pp. 1285–1298.

[10] S. Ying, B. Wang, L. Wang, Q. Li, Y. Zhao, J. Shang, H. Huang, G. Cheng, Z. Yang, and J. Geng, "An improved knn-based efficient log anomaly detection method with automatically labeled samples," *ACM Trans. Knowl. Discov. Data*, vol. 15, no. 3, pp. 34:1–34:22, 2021.

[11] Q. Zhang, H. Wu, C. Zhang, Q. Hu, H. Fu, J. T. Zhou, and X. Peng, "Provable dynamic fusion for low-quality multimodal data," in *ICML*, vol. 202, 2023, pp. 41 753–41 769.

[12] C. Hou, T. Jia, Y. Wu, Y. Li, and J. Han, "Diagnosing performance issues in microservices with heterogeneous data source," in *ISPA/BDCloud/SocialCom/SustainCom*, 2021, pp. 493–500.

[13] P. Wang, X. Zhang, Z. Cao, and Z. Chen, "MADMM: microservice system anomaly detection via multi-modal data and multi-feature extraction," *Neural Comput. Appl.*, vol. 36, no. 25, pp. 15 739–15 757, 2024.

[14] F. Liu, Y. Wang, Z. Li, R. Ren, H. Guan, X. Yu, X. Chen, and G. Xie, "Microcbr: Case-based reasoning on spatio-temporal fault knowledge graph for microservices troubleshooting," in *ICCBR*, vol. 13405, 2022, pp. 224–239.

[15] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, and D. Zhang, "Deeptralog: Trace-log combined microservice anomaly detection through graph-based deep learning," in *ICSE*, 2022, pp. 623–634.

[16] S. Zhang, P. Jin, Z. Lin, Y. Sun, B. Zhang, S. Xia, Z. Li, Z. Zhong, M. Ma, W. Jin, D. Zhang, Z. Zhu, and D. Pei, "Robust failure diagnosis of microservice system through multimodal data," *IEEE Trans. Serv. Comput.*, vol. 16, no. 6, pp. 3851–3864, 2023.

[17] G. Yu, P. Chen, Y. Li, H. Chen, X. Li, and Z. Zheng, "Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data," in *ESEC/FSE*, 2023, pp. 553–565.

[18] C. Zhao, M. Ma, Z. Zhong, S. Zhang, Z. Tan, X. Xiong, L. Yu, J. Feng, Y. Sun, Y. Zhang, D. Pei, Q. Lin, and D. Zhang, "Robust multimodal failure detection for microservice systems," in *KDD*, 2023, pp. 5639–5649.

[19] J. Huang, Y. Yang, H. Yu, J. Li, and X. Zheng, "Twin graph-based anomaly detection via attentive multi-modal learning for microservice system," in *ASE*, 2023, pp. 66–78.

[20] Y. Wang, Z. Zhu, Q. Fu, Y. Ma, and P. He, "MRCA: metric-level root cause analysis for microservices via multi-modal data," in *ASE*, 2024, pp. 1057–1068.

[21] N. Zhao, J. Chen, Z. Yu, H. Wang, J. Li, B. Qiu, H. Xu, W. Zhang, K. Sui, and D. Pei, "Identifying bad software changes via multimodal anomaly detection for online service systems," in *ESEC/FSE*, 2021, pp. 527–539.

[22] H. Liu, X. Huang, M. Jia, T. Jia, J. Han, Z. Wu, and Y. Li, "UAC-AD: unsupervised adversarial contrastive learning for anomaly detection on multi-modal data in microservice systems," *IEEE Trans. Serv. Comput.*, vol. 17, no. 6, pp. 3887–3900, 2024.

[23] X. Jiang, H. Luo, Y. Sun, and M. Guizani, "Fast anomaly detection for iot services based on multisource log fusion," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 9405–9419, 2024.

[24] C. Lee, T. Yang, Z. Chen, Y. Su, and M. R. Lyu, "Eadro: An end-to-end troubleshooting framework for microservices on multi-source data," in *ICSE*, 2023, pp. 1750–1762.

[25] C. Wu, P. Chen, B. Yin, Z. Lin, C. Jiang, D. Yu, C. Zou, and C. Lui, "Using adamic-adar index algorithm to predict volunteer collaboration: Less is more," *CoRR*, vol. abs/2308.13176, 2023.

[26] P. Ni, R. Okhrati, S. Guan, and V. Chang, "Knowledge graph and deep learning-based text-to-graphql model for intelligent medical consultation chatbot," *Inf. Syst. Frontiers*, vol. 26, no. 1, pp. 137–156, 2024.

[27] P. Lai, F. Ye, Y. Fu, Z. Chen, Y. Wu, Y. Wang, and V. Chang, "Cognlg: Cognitive graph for kg-to-text generation," *Expert Syst. J. Knowl. Eng.*, vol. 41, no. 1, 2024.

[28] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *AAAI/IAAI/EAAI*, 2021, pp. 4027–4035.

[29] S. Han and S. S. Woo, "Learning sparse latent graph representations for anomaly detection in multivariate time series," in *KDD*, 2022, pp. 2977–2986.

[30] Z. Chen, D. Chen, X. Zhang, Z. Yuan, and X. Cheng, "Learning graph structures with transformer for multivariate time-series anomaly detection in iot," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9179–9189, 2022.

[31] S. He, G. Li, K. Xie, and P. K. Sharma, "Fusion graph structure learning-based multivariate time series anomaly detection with structured prior knowledge," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 8760–8772, 2024.

[32] M. Du, F. Li, G. Zheng, and V. Srikumar, "Deeplog: Anomaly detection and diagnosis from system logs through deep learning," in *CCS*, 2017, pp. 1285–1298.

[33] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, Y. Chen, R. Zhang, S. Tao, P. Sun, and R. Zhou, "Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *IJCAI*, 2019, pp. 4739–4745.

[34] S. Nedelkoski, J. Bogatinovski, A. Acker, J. Cardoso, and O. Kao, "Self-attentive classification-based anomaly detection in unstructured logs," in *ICDM*, 2020, pp. 1196–1201.

[35] S. Xue, H. Chen, and X. Zheng, "Detection and quantification of anomalies in communication networks based on LSTM-ARIMA combined model," *Int. J. Mach. Learn. Cybern.*, vol. 13, no. 10, pp. 3159–3172, 2022.

[36] V. Le and H. Zhang, "Log-based anomaly detection without log parsing," in *ASE*. IEEE, 2021, pp. 492–504.

[37] S. He, Y. Lei, Y. Zhang, K. Xie, and P. K. Sharma, "Parameter-efficient log anomaly detection based on pre-training model and LORA," in *ISSRE*, 2023, pp. 207–217.

[38] P. Liu, H. Xu, Q. Ouyang, R. Jiao, Z. Chen, S. Zhang, J. Yang, L. Mo, J. Zeng, W. Xue, and D. Pei, "Unsupervised detection of microservice trace anomalies through service-level deep bayesian networks," in *ISSRE*, 2020, pp. 48–58.

[39] T. Zhou, C. Zhang, X. Peng, Z. Yan, P. Li, J. Liang, H. Zheng, W. Zheng, and Y. Deng, "Tracestream: Anomalous service localization based on trace stream clustering with online feedback," in *ISSRE*, 2023, pp. 601–611.

[40] C. Lee, T. Yang, Z. Chen, Y. Su, Y. Yang, and M. R. Lyu, "Heterogeneous anomaly detection for software systems via semi-supervised cross-modal attention," in *ICSE*, 2023, pp. 1724–1736.

[41] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *ICWS*, 2017, pp. 33–40.

[42] I. Elsharkawi, H. Sharara, and A. Rafea, "Svig: A similarity-thresholded approach for vision graph neural networks," *IEEE Access*, vol. 13, pp. 19 379–19 387, 2025.

[43] R. Ghiasi, H. Amirkhani, and A. Bosaghzadeh, "Multi-view graph structure learning using subspace merging on grassmann manifold," *Multim. Tools Appl.*, vol. 82, no. 11, pp. 17 135–17 157, 2023.

[44] C. Shang, J. Chen, and J. Bi, "Discrete graph structure learning for forecasting multiple time series," in *ICLR*, 2021.

[45] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *KDD*, 2019, pp. 2828–2837.

[46] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly transformer: Time series anomaly detection with association discrepancy," in *ICLR*, 2022.

[47] X. Li, P. Chen, L. Jing, Z. He, and G. Yu, "Swisslog: Robust anomaly detection and localization for interleaved unstructured logs," *IEEE Trans. Dependable Secur. Comput.*, vol. 20, no. 4, pp. 2762–2780, 2023.

[48] S. Yan, Y. Xiong, and D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition," in *AAAI*, 2018, pp. 7444–7452.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 23,2026 at 21:31:17 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.